

03 · Geometry Graph 设计

2026-07-31

Contents

03 · Geometry Graph 设计	1
1. 设计目标	1
2. 形式化定义	2
3. 节点类型	2
3.1 节点 Schema (Pydantic)	2
4. 边 (关系) 类型	3
4.1 边 Schema	4
5. JSON 示例	4
6. 图构建算法	4
6.1 总体流程	4
6.2 节点注册伪代码	5
6.3 边构建伪代码	5
6.4 后处理：交点回填与多边形识别	5
6.5 复杂度控制	5
7. 查询接口	6
7.1 实现示例	6
8. 增量更新	6
9. 持久化与序列化	7
10. 与下游模块的契约	7
11. 设计路线小结	7

03 · Geometry Graph 设计

本文档阐述 Geometry Graph 的形式化定义、Schema、构建算法、查询接口、增量更新与持久化。Graph 是连接感知与推理的核心数据结构，是 Geometry World Model 的载体。

1. 设计目标

Geometry Graph 以图的形式显式表示题目中所有几何对象及其关系。其设计目标：

- 1. **完备性**：覆盖中考、高考全部几何对象与关系。
- 2. **可计算**：节点携带几何度量，边携带可验证属性。
- 3. **可查询**：提供面向 LLM 与 Solver 的高层查询 API。
- 4. **可演进**：支持验证状态、置信度、多假设标记。
- 5. **可持久化**：JSON 序列化，便于调试与回放。

2. 形式化定义

Geometry Graph 定义为带标签多重有向图：

$$G = (V, E, \tau_V, \tau_E, \mu_V, \mu_E)$$

- V ：节点集合（几何对象）。
- $E \subseteq V \times V$ ：边集合（关系）。
- $\tau_V : V \rightarrow \text{NodeType}$ ：节点类型函数。
- $\tau_E : E \rightarrow \text{RelType}$ ：边类型函数。
- $\mu_V : V \rightarrow \text{Attrs}$ ：节点属性（坐标、方程、半径等）。
- $\mu_E : E \rightarrow \text{Attrs}$ ：边属性（confidence、verified、angle、evidence 等）。

采用**有向多重图** (MultiDiGraph) 而非简单图，原因：同一对节点可存在多种关系（如 On 与 Tangent 同时成立），需以多重边表达。

3. 节点类型

Node 类型	示例	必备属性	可选属性
Point	Point(A)	coords	label, source, confidence
Line	Line(AB)	equation/direction	label
Segment	Segment(AB)	endpoints, length	label, equation
Ray	Ray(A, dir)	origin, direction	label
Circle	Circle(0)	center, radius	label, fit_residual
Arc	Arc(0, A, B)	center, radius, start_angle, end_angle	label
Ellipse	Ellipse(E)	center, semi_major, semi_minor, rotation	foci, eccentricity, label
Polygon	Triangle(ABC)	vertices	type(triangle/quad/...), area

3.1 节点 Schema (Pydantic)

```
from pydantic import BaseModel
from typing import Literal, Optional

class PointNode(BaseModel):
    id: str
    type: Literal["Point"] = "Point"
    label: Optional[str]
    coords: tuple[float, float]
    source: Literal["corner", "endpoint", "intersection", "explicit", "inferred"]
```

```
confidence: float
```

```
class CircleNode(BaseModel):
```

```
    id: str
```

```
    type: Literal["Circle"] = "Circle"
```

```
    label: Optional[str]
```

```
    center: tuple[float, float]
```

```
    radius: float
```

```
    fit_residual: float
```

```
    coverage: float # 1.0= 完整圆, <1.0= 弧
```

```
    confidence: float
```

```
class EllipseNode(BaseModel):
```

```
    id: str
```

```
    type: Literal["Ellipse"] = "Ellipse"
```

```
    label: Optional[str]
```

```
    center: tuple[float, float]
```

```
    semi_major: float
```

```
    semi_minor: float
```

```
    rotation: float
```

```
    foci: list[tuple[float, float]]
```

```
    eccentricity: float
```

```
    confidence: float
```

```
# ... Line/Segment/Ray/Arc/Polygon 类似
```

4. 边（关系）类型

关系	符号	定义	方向	典型容差
On / LiesOn	$\in$	点在直线/线段/圆/椭圆上	Point→Obj	距离 < 2px 或 < 1% 尺度
Center	—	点为圆心	Point→Circle	圆心重合
Collinear	—	三点及以上共线	Point→Point(组)	点到直线距离 < 容差
Intersect	$\cap$	两线/圆/椭圆相交	Obj→Obj	解析求交存在
Tangent	—	直线/圆与圆相切	Obj→Circle	距离 = 半径, 容差内
Parallel	//	两直线方向平行	Line→Line	夹角 < 3°
Perpendicular	$\perp$	两直线方向垂直	Line→Line	夹角 $\in [87^\circ, 93^\circ]$
Equal	$\equiv$	两线段等长/两角相等	Obj→Obj	相对误差 < 2%
Inside	—	点在圆/多边形内部	Point→Obj	解析判定
Outside	—	点在圆/多边形外部	Point→Obj	解析判定
Concentric	—	两圆同心	Circle→Circle	圆心距 < 容差
TangentPoint	—	切线与圆的切点	Point→Circle	On + Tangent 联合
Inscribed	—	多边形内接于圆	Polygon→Circle	顶点全在圆上
Circumscribed	—	多边形外切于圆	Polygon→Circle	边均与圆相切
SameArc	—	两弧同弧	Arc→Arc	同圆同覆盖
Similar	$\propto$	两三角形相似	Polygon→Polygon	对应角相等

关系	符号	定义	方向	典型容差
Congruent	$\cong$	两三角形全等	Polygon→Polygon	对应边相等

4.1 边 Schema

```
class Edge(BaseModel):
    src: str          # 源节点 id
    dst: str          # 目标节点 id
    rel: RelType
    confidence: float
    verified: Literal["true","false","uncertain","pending"] = "pending"
    evidence: Optional[str]    # 验证证据字符串
    attrs: dict = {}          # 关系特有属性 (angle, tangent_point 等)
```

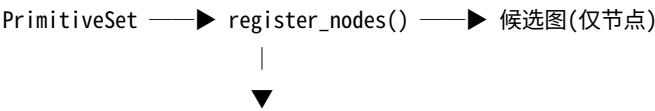
5. JSON 示例

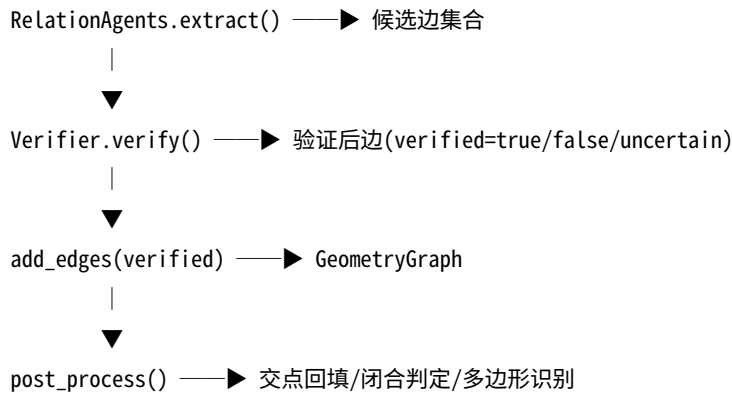
对应” 点 A 在圆 O 上, AB 切圆 O 于 A, OA⊥AB”:

```
{
  "graph_version": "1.0",
  "nodes": [
    {"id": "P_A", "type": "Point", "label": "A", "coords": [180.0, 84.5], "confidence": 0.97},
    {"id": "P_B", "type": "Point", "label": "B", "coords": [260.0, 140.0], "confidence": 0.96},
    {"id": "P_O", "type": "Point", "label": "O", "coords": [180.0, 160.0], "confidence": 0.99},
    {"id": "C_O", "type": "Circle", "label": "O", "center": [180.0, 160.0], "radius": 75.5, "confidence": 0.96},
    {"id": "L_AB", "type": "Segment", "label": "AB", "endpoints": [[180.0, 84.5], [260.0, 140.0]], "length": 98.1, "confidence": 0.94},
    {"id": "L_OA", "type": "Segment", "label": "OA", "endpoints": [[180.0, 160.0], [180.0, 84.5]], "length": 75.5, "confidence": 0.95}
  ],
  "edges": [
    {"src": "P_A", "dst": "C_O", "rel": "On", "confidence": 0.98, "verified": "true", "evidence": "|dist-r|=0.2<tol"},
    {"src": "L_AB", "dst": "C_O", "rel": "Tangent", "tangent_point": "P_A", "confidence": 0.95, "verified": "true", "evidence": "d=75.4≈r"},
    {"src": "L_OA", "dst": "L_AB", "rel": "Perpendicular", "angle": 90.0, "confidence": 0.96, "verified": "true", "evidence": "θ=90.12°"},
    {"src": "P_A", "dst": "L_AB", "rel": "On", "confidence": 1.0, "verified": "true"},
    {"src": "P_O", "dst": "C_O", "rel": "Center", "confidence": 1.0, "verified": "true"}
  ],
  "metadata": {"image_size": [400, 320], "scale_px_per_cm": 12.0}
}
```

6. 图构建算法

6.1 总体流程





6.2 节点注册伪代码

```

def register_nodes(primitives, graph):
    for p in primitives.points:
        graph.add_node(p.id, type="Point", label=p.label, coords=p.coords, ...)
    for s in primitives.segments:
        graph.add_node(s.id, type="Segment", endpoints=s.endpoints, ...)
    for c in primitives.circles:
        graph.add_node(c.id, type="Circle", center=c.center, radius=c.radius, ...)
    # ... 其他类型

```

6.3 边构建伪代码

```

def add_verified_edges(candidates, graph, verifier):
    for cand in candidates:
        result = verifier.verify(cand) # 返回 verified/evidence/attrs
        if result.verified == "true":
            graph.add_edge(cand.src, cand.dst, rel=cand.rel,
                           confidence=cand.confidence, verified="true",
                           evidence=result.evidence, attrs=result.attrs)
        elif result.verified == "uncertain":
            graph.add_edge(cand.src, cand.dst, rel=cand.rel,
                           confidence=cand.confidence*0.5, verified="uncertain",
                           evidence=result.evidence)
    # false 丢弃

```

6.4 后处理：交点回填与多边形识别

- **交点回填**：对 Intersect 边，计算解析交点，若该点不在已注册 Point 中，新增 Point 节点（source=inferred）。
- **闭合判定**：若一组 Segment 首尾相连形成闭合环，识别为 Polygon，新增 Polygon 节点，顶点为环上 Point。
- **多边形类型**：3 顶点 → Triangle，4 → Quadrilateral。

6.5 复杂度控制

朴素关系判定为 $O(n^2)$ （每对节点）。优化：- **空间分桶**：点按网格分桶，仅与邻近桶的对象判定 $O(n)$ 。- **类型过滤**：仅对类型兼容的对调用相应判定（如 $O(n)$ 仅 Point → Obj）。- **候选预筛**：用包围盒快速排除远距离对。

7. 查询接口

提供面向 LLM 与 Solver 的高层查询 API（封装 NetworkX）：

接口	签名	返回
neighbors	neighbors(node, rel=None)	满足某关系的邻居节点
points_on	points_on(obj_id)	某线/圆上的所有点
circles_through	circles_through(point_id)	过某点的所有圆
tangent_lines	tangent_lines(circle_id)	某圆的所有切线
tangent_points	tangent_points(circle_id)	某圆的所有切点
lines_through	lines_through(point_id)	过某点的所有线
intersection	intersection(obj1, obj2)	两对象交点
path_constraints	path_constraints(p, q)	p 到 q 的约束链
subgraph	subgraph(node_ids)	诱导子图
all_verified	all_verified(rel)	所有已验证的某类关系

7.1 实现示例

```
class GeometryGraph:
    def __init__(self):
        self.g = nx.MultiDiGraph()

    def points_on(self, obj_id):
        return [n for n in self.g.predecessors(obj_id)
                if self.g[n][obj_id] and any(
                    d["rel"]=="On" and d["verified"]=="true"
                    for d in self.g[n][obj_id].values())]

    def tangent_lines(self, circle_id):
        res = []
        for n, _, d in self.g.in_edges(circle_id, data=True):
            if d.get("rel")=="Tangent" and d.get("verified")=="true":
                res.append(n)
        return res

    def to_dsl(self):
        from ..dsl.serializer import to_dsl
        return to_dsl(self)
```

8. 增量更新

LLM 推理过程中会提出新假设（如“ $\triangle ABE \sim \triangle ACD$ ”），Solver 推导出新关系。Graph 需支持增量更新：

操作	触发	处理
add_node	LLM 引入辅助点/线	新增节点，标记 source=constructed

操作	触发	处理
add_edge(verified)	Solver 推导	新增边, verified=true, source=derived
add_edge(hypothesis)	LLM 假设	新增边, verified=pending, 待 Verifier 验证
reject_edge	Verifier 否定	删除或标记 verified=false

增量更新记录到 derivation_log，使证明链可追溯。

9. 持久化与序列化

- **JSON 序列化**: to_json() / from_json(), 用于调试、回放、数据集存储。
- **DSL 序列化**: to_dsl() / from_dsl(), 用于 LLM Prompt (见 06-DSL)。
- **图快照**: 每个推理阶段保存 Graph 快照，支持回溯调试。

10. 与下游模块的契约

下游	使用方式
Verifier	读取候选边，写回 verified 字段
DSL Serializer	读取已验证边，序列化为 DSL
LLM Agent	通过查询接口 + DSL 获取图信息
Solver	通过查询接口读取度量，回写推导边

11. 设计路线小结

Geometry Graph 的设计路线为：“**形式化定义图结构** → **Pydantic Schema 约束** → **节点注册** → **候选边抽取** → **验证入图** → **后处理补全** → **高层查询 API** → **增量更新支持推导**”。它以 NetworkX 多重有向图为底座，以验证状态为核心字段，使几何世界模型既完备又可计算，成为感知与推理之间的可靠桥梁。